FORWARD DEPLOYED ENGINEER | CASE STUDY

Improving the MoveInSync Employee App Rating

Evidence from 1,000 collected Google Play reviews, and a 30-day execution plan

Prepared by Tharun Teja
10 October 2026

Abstract

The MoveInSync employee app shows a 4.3 star rating on Google Play from about 1.15 lakh ratings (noted by hand on 10 October 2026). That number says how people feel on average, but not what to fix. To get closer to the causes, I wrote a Google Colab notebook that collected the newest 1,000 publicly accessible reviews of the app (India store, 14 June to 9 October 2026). I labelled the 292 one-star and two-star reviews with keyword rules and compared the results with five reviews that I read by hand.

In this sample 29.2% of reviews are one or two stars, and one-star reviews make up almost 89% of those. Among the low-star reviews, the largest named theme is driver and vehicle operations (about 25% of low-star reviews mention it), followed by booking and scheduling (about 22%), crashes, speed and network (about 17%) and GPS and ETA (about 13%). About 38% have no clear theme. Written reviews arrive at a steady 8 to 9 a day, so the drop in October is only because the month is incomplete.

I do not claim to know the root causes. I treat these results as a ranked list of leads. My recommendation is to make the commute reliable first and let the rating follow. Section 5 turns that into a 30-day plan with owners, steps, release rules and measures.

Main findings

- Low-star reviews are 29.2% of the sample (292 of 1,000). Of those, 259 are one-star.
- The sample average is 3.77 stars, lower than the listing score of 4.3. I do not read this as a decline, for the reasons in Section 3.1.
- Driver and vehicle operations, booking and scheduling, crashes and network, and GPS and ETA are the main named themes. Part of the pain is operational and not a software defect.
- No-show and check-in complaints are rare (about 3%) but can cost an employee money, so I rank them by consequence and not by count.
- Review volume per day is flat across the months we can see. There is no sign of a change in either direction.

Item	Detail
App	MoveInSync employee app for Android (com.moveinsync.ets)
Listing check	4.3 stars, about 1.15 lakh ratings, India view. App update dated 8 October 2026. Noted by hand on 10 October 2026
Main data	1,000 unique Google Play reviews, 14 June to 9 October 2026, of which 292 are one-star or two-star
Other sources	MoveInSync Help Center pages, five reviews read by hand, and the case brief and job description I was given

Item	Detail
Data used	Public data only. No internal MoveInSync data or private employee data was used

1. Introduction and research questions

People use this app early in the morning, when they are trying to reach work on time. If a booking is wrong or the vehicle is missing from the map, they lose time, and sometimes money. The case brief says the rating needs to improve, so I started from there. One average number cannot show which problems matter most or whether the app is the cause, so I asked four questions:

- **RQ1.** What do unhappy reviewers complain about, and how big is each theme?
- **RQ2.** Is the number of reviews changing over the period we can see?
- **RQ3.** Which problems look like app problems, and which look like operations problems?
- **RQ4.** What should the first 30 days of work be, and how will we know it worked?

2. Data and method

2.1 How the data was collected

I wrote a notebook, MoveInSync_Review_Collection_and_Analysis.ipynb, that runs in Google Colab. It asks for the newest accessible Google Play review pages for com.moveinsync.ets (India store, English), up to 1,000 records. It removes duplicates by review ID or fingerprint and saves one combined CSV.

Step	What the script does	Why
1. Target	Package com.moveinsync.ets, India store, English	A clear and repeatable source
2. Collect	Requests the newest accessible review pages, up to 1,000 records	Gets the most recent written reviews
3. Remove duplicates	Keeps unique review IDs or fingerprints. Result: 1,000 unique records	So no review is counted twice
4. Save	Writes one CSV: rating, date, review text, review length, app version when available, source URL and rule-based theme labels	So others can check and re-run the analysis
5. Label	Applies keyword rules to the one-star and two-star records (292), and allows several themes per review	Turns text into counts
6. Context pages	Reads the Google Play listing and three MoveInSync Help Center pages (all returned HTTP 200)	Public background only. These pages say nothing about how common any problem is

Table 1. The collection steps.

Open the notebook:
https://colab.research.google.com/drive/1fP52lR-nGd6N0qryWATvWfgnzOhGsO0j?usp=sharing

To run it again, open the link and choose Runtime, then Run all. The saved CSV lists reviews but not reviewer names among its fields. I would still remove personal details from any review text before sharing it outside the team.

2.2 A problem in the collection, and what I did about it

The notebook also tried a separate pull of one-star and two-star reviews. That step failed. The installed scraper expects the argument `filter_score_with`, but my notebook used `filter_score`, so no extra low-star records came back. This matters for how to read the results. The 292 low-star reviews in this report are the low-star part of the 1,000-review all-rating sample. They are not a separate targeted sample, so the low-star group is smaller than I planned. I have put the fix and a re-run into Phase 0 of the plan.

2.3 How the reviews were labelled

Each one-star and two-star review was labelled with keyword rules into these themes: driver and vehicle operations; booking and scheduling; crashes, performance and network; GPS, ETA and tracking; UX and usability; notifications and alerts; customer support; login and access; no-show and check-in; and other or unclear. The labels are multi-label, so one review can count under several themes. The chart and table therefore count *mentions*, not separate reviews, and the percentages should not be added up. The rules have not yet been checked by a person, so I treat each label as a hint to look closer.

2.4 The listing score

The 4.3 stars, the 1.15 lakh ratings and the 8 October update date come from the India listing, which I read by hand on 10 October 2026. The script's saved notes did not capture them. The notebook therefore cannot confirm the current score on its own, and I would re-capture these three values, with the date and time, before anyone quotes them as current.

2.5 What could go wrong with this data

- It is a convenience sample: the newest 1,000 reviews that were accessible. It covers 118 days, not the history of the app.
- It contains written reviews only. People who write are not a fair cross-section of all employees, and unhappy people write more often.
- The keyword labels are not validated, and about 38% of low-star reviews have no clear theme.
- A review mentioning a theme shows what the person says. It does not prove the cause.
- Counts have no trip or user numbers behind them, so they are not rates of failure.
- I rely on a community scraper, not an official Google API. It can change or break, as the star filter did.
- I do not give confidence intervals. The sample is not random, so an interval would suggest more precision than the data has.

3. Results

3.1 Ratings in the sample (RQ1)

Table 2 and Figure 1 show the star ratings. Of the 1,000 reviews, 292 (29.2%) are one-star or two-star, and 259 of those 292 (88.7%) are one-star.

Stars	Reviews	Share of sample
1 star	259	25.9%
2 stars	33	3.3%
3 stars	26	2.6%

Stars	Reviews	Share of sample
4 stars	42	4.2%
5 stars	640	64.0%
Total	1,000	100%

Table 2. Star ratings in the collected sample.

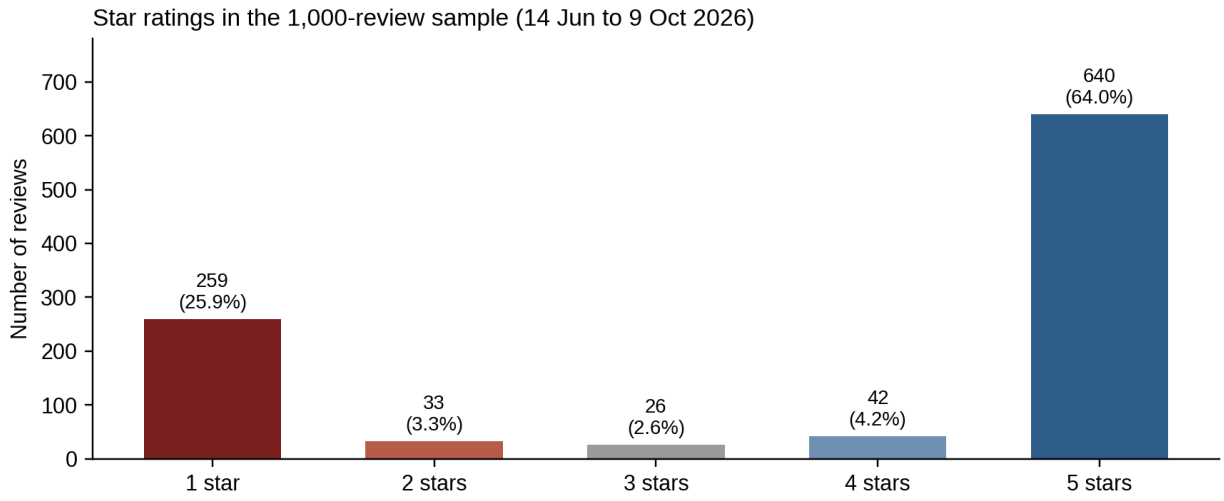


Figure 1. Star ratings in the 1,000-review sample.

The sample is mostly positive: 64% are five-star. The average works out to 3.77 stars (from Table 2), which is lower than the listing score of 4.3. I would not read that gap as the rating falling. The listing score covers all ratings over the life of the app, and I believe it includes people who only tapped a star without writing. My sample covers only recent written reviews, and people who take the time to write are more likely to be unhappy. To test whether sentiment is really falling, we would need the history of the rating, or the monthly mix of stars over a longer window. The notebook can produce that if we run it on more history.

3.2 What unhappy reviewers complain about (RQ1, RQ3)

Figure 2 is the chart the notebook produced. I read the bar lengths against the axis to make Table 3, so the counts are close but not exact. The last column divides the mentions by the 292 low-star reviews. Because a review can carry several labels, the rows overlap and cannot be added.

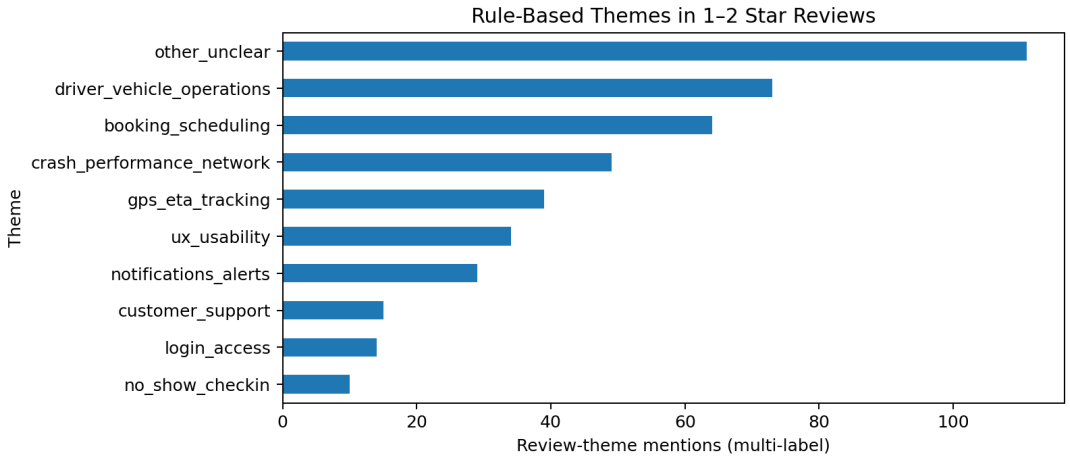


Figure 2. Rule-based themes in the one-star and two-star reviews (multi-label, so these are mentions).

Theme	Mentions (approx.)	Per 100 low-star reviews (approx.)
other_unclear	111	38
driver_vehicle_operations	73	25
booking_scheduling	64	22
crash_performance_network	49	17
gps_eta_tracking	39	13
ux_usability	34	12
notifications_alerts	29	10
customer_support	15	5
login_access	14	5
no_show_checkin	10	3

Table 3. Theme mentions read from Figure 2. Chart readings: please confirm against the CSV.

What I take from this:

- Driver and vehicle operations is the largest named theme, mentioned in about one low-star review in four. This points at operations (late, missing or changed vehicles, driver behaviour) more than at software, and it means Operations has to be part of the fix from the start.
- Booking and scheduling comes next, at about 22%. It matches the booking complaints in the reviews I read by hand.
- Crashes, speed and network are about 17%. Some of that may be the user's phone or signal, but it shows that basic reliability matters.
- GPS and ETA (about 13%) and notifications (about 10%) are smaller, but they hit the moment when someone is standing outside waiting for a ride.
- No-show and check-in is only about 3%. It is rare, but a wrong no-show can cost an employee money, so I rank it by consequence and not by count.
- About 38% have no clear theme. If the rules give “other_unclear” only when nothing else matches (I need to confirm this in the code), then about 181 of the 292 reviews, or 62%, carry at least one specific theme. Reading a sample of the unclear group is the first improvement to make.

3.3 Review timing (RQ2)

Figure 3 is the notebook's chart of monthly counts: June 150, July 286, August 241, September 245 and October 78. Read alone, it looks like a rise in July and a sharp fall in October. That would be a mistake, because the 1,000-review cap means the window is not made of whole months.

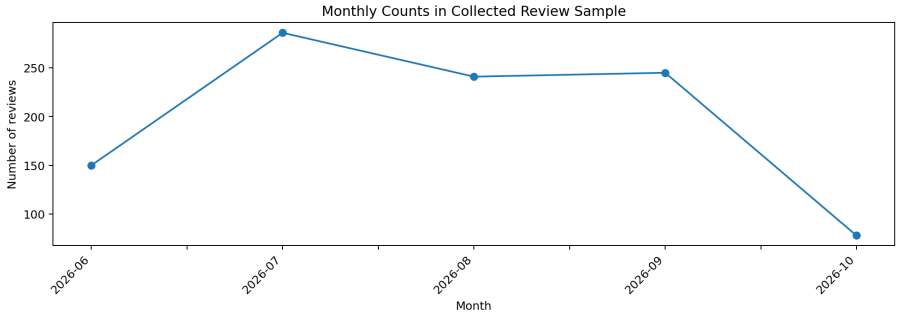


Figure 3. Monthly counts of collected reviews (from the notebook).

The window runs from 14 June to 9 October, so June has only 17 days and October only 9. When I divide each month by the days it covers, the picture is very different.

Month	Days covered	Reviews	Reviews per day
June (from the 14th)	17	150	8.8
July	31	286	9.2
August	31	241	7.8
September	30	245	8.2
October (to the 9th)	9	78	8.7
Whole window	118	1,000	8.5

Table 4. Reviews per day, by month. Assumes the collection has no gaps in the window.

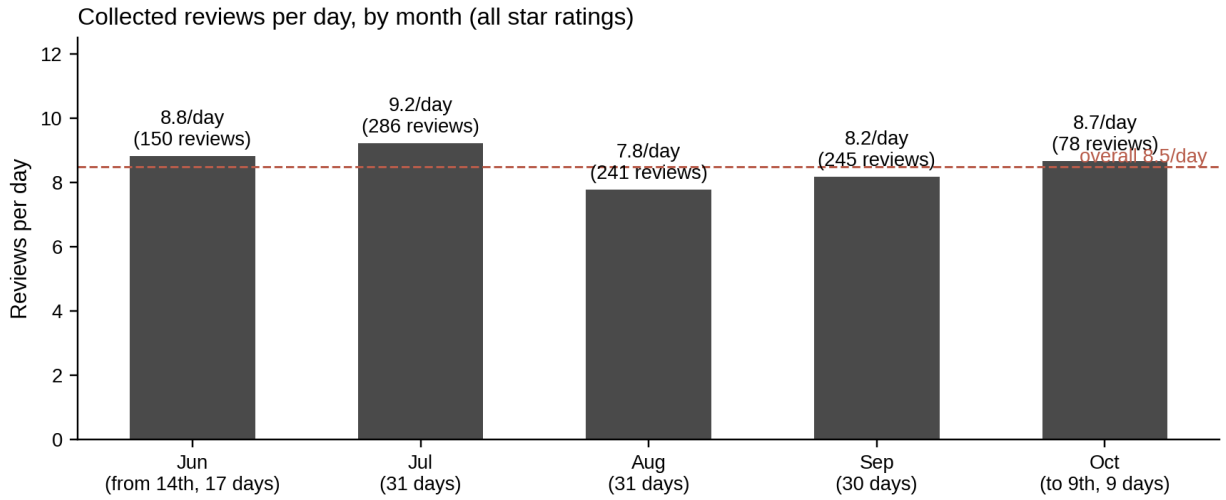


Figure 4. Reviews per day. The rate stays between about 7.8 and 9.2.

The inflow is steady at roughly 8 to 9 written reviews a day, and about 2.5 low-star reviews a day. The fall in October disappears once the short month is taken into account. So these counts do not show that the app got worse in July or better in October. They also depend on users, trips, releases and collection coverage, so the right next step is to mark releases on a timeline and compare low-star reviews with trip numbers.

3.4 Reviews I read by hand

Before building the notebook I read five reviews on the public listing. I use them only to give a feel for the themes. I never use them to count anything. R01 is dated before the window of the dataset. I did not check whether the other four also appear in the CSV.

ID / date	What the reviewer described	Theme	Why it matters
R01 **11 Jun 2026**	Unreliable driver GPS, wrong ETA and pickup time, missed pickups, delayed or missing alerts	GPS/ETA; alerts	Hits the \u201cwhere is my ride\u201d moment
R02 **18 Jun 2026**	Bookings made for different dates; a no-show status despite a claimed booking; a trip-rating prompt; worry about a salary deduction	Booking; no-show	Possibly costly. Check first

ID / date	What the reviewer described	Theme	Why it matters
R03 29 Jun 2026	Suggests a check-in reminder once the vehicle passes the pickup point, because a missed check-in can become a surprise no-show	Check-in; alerts	Good idea, and a sign of confusion
R04 1 Jul 2026	Slow GPS, often wrong pickup and drop times, vehicle location not visible	GPS/ETA	Tracking trust
R05 7 Jul 2026	A shuttle booking went from one step to two, and the reviewer is not sure a cab was booked	Booking screen	Possible confirmation doubt

Table 5. Five reviews read by hand (illustration only).

3.5 Putting the two sources together

Question	Collected data (low-star reviews)	Reviews read by hand	My reading
Booking problems?	About 22% mention booking	R02, R05	Both sources agree. First place to trace real cases
Tracking and ETA?	About 13% mention GPS/ETA	R01, R04	Both agree. Measure how old locations are and how wrong ETAs are
Alerts?	About 10% mention alerts	R01, R03	Present. Likely to overlap with other themes
No-show and check-in?	About 3%	R02, R03	Rare in counts, high in consequence. Audit the rules
Crashes and sign-in?	About 17% crashes/network, about 5% login	None	New signal from the larger data. Keep it running alongside
Driver and vehicle?	About 25%, the largest named theme	Indirect (late vehicles)	Mostly operational. Involve Operations early

Table 6. The two sources compared.

3.6 What I can and cannot claim

The data supports	The data does not support
Low-star reviews are 29.2% of this recent written sample, mostly one-star.	That a theme caused the overall rating.
Driver and vehicle operations, booking, crashes and network, and GPS/ETA are the biggest named themes (approximate).	How often trips fail, or which app versions or sites are affected.
Written reviews arrive at a steady 8 to 9 a day in this window.	Any trend in sentiment, because the window is only 118 days and has no trip numbers.
The listing score and the sample average differ, and the reasons are plausible.	Whether a complaint is an app bug or an operations problem. That needs internal data.

Table 7. Limits of the evidence.

4. What I think is going wrong

The case brief describes the full journey: onboarding, roster set-up, booking, vehicle assignment and routing, pickup time messages, live tracking, OTP and driver safety checks, finishing the trip, and raising issues. The app is where most of that shows up for the employee. Table 8 matches the themes to those steps.

Journey step	What the employee needs	Matching theme (approx. per 100 low-star reviews)
1. Book	The right date, time and cab or shuttle, with a clear confirmation	Booking and scheduling (22); UX (12)
2. Get a vehicle and be told	A vehicle, a pickup time and alerts on time	Driver and vehicle operations (25); alerts (10)
3. Wait and board	A fresh vehicle location, a trustworthy ETA, and a clear check-in and no-show status	GPS and ETA (13); no-show and check-in (3)
4. Finish the trip	A correct trip record, and a way to raise problems	Customer support (5)
All the time	An app that opens and signs in	Crashes, speed and network (17); login (5)

Table 8. Themes matched to the commute journey.

My main idea has two parts. First, many complaints are cases where the app and the real trip do not agree: what was booked, where the vehicle is, when it will arrive, and whether an alert came in time. I call this **state correctness and journey visibility**. Second, the single largest named theme is about drivers and vehicles, so a good part of the problem may be operational. Both parts have to be tested with internal event data.

Other explanations I need to rule out

- A late vehicle, a weak GPS signal or a poor network may be an operations or connection problem and not an app bug.
- A booking screen may have been changed on purpose but not explained well. Usability tests can tell confusing design apart from a booking that really failed to save.
- An alert can leave the server and still never be seen, because of phone permissions, battery settings or network. Sent is not the same as seen.
- Many complaints may come from one site or one shift group and not from everyone.
- A rule-based label may be wrong. Complaints that look like a booking problem could be about something else.

Idea to test	What the data shows (approx.)	Data to look at	How to test it	Order
H1. What the employee booked and what was saved can differ	Booking: about 22% of low-star reviews; R02, R05	Chosen date, time and type; booking ID; confirmation; retries; the saved record	Follow sample cases from the tap to the saved trip. Compare what the user chose, what was saved, what the screen said and which trip was assigned	P0
H2. No-show or check-in status can be a surprise or wrongly applied	About 3%; R02, R03. Rare but costly	Status changes, check-in events, waiting rules, GPS near pickup, driver timestamps, disputes	Review every status change. Count the no-shows that were wrong	P0
H3. Vehicle location and ETA can be old or wrong	GPS/ETA: about 13%; R01, R04	Time a GPS point was made versus received, location age, ETA error, device, network	Measure how old locations are and how far ETAs are off, by trip, route and time	P1

Idea to test	What the data shows (approx.)	Data to look at	How to test it	Order
H4. Important alerts can be late or not useful	Alerts: about 10%; R01, R03	Trigger time, push response, device token, retries, permissions	Measure delay at each step and find where alerts fail. Test an in-app trip timeline as a backup	P1
H5. A changed booking screen makes people unsure	UX: about 12%; R05	Steps completed, taps, confirmation screen, drop-offs	Short usability tests. Compare before and after on a small group	P2
H6. Crashes, slow screens or sign-in problems stop people using the app	Crashes, speed, network: about 17%; login: about 5%	OTP success, login time, crashes, launch success, phone model, OS, app version	Break down failures and check whether they match low-star reviews	P1
H7. Late, missing or changed vehicles and driver problems drive many complaints	Driver and vehicle operations: about 25%, the largest named theme	Scheduled versus actual pickup time, vehicle and driver changes, cancellations, by route and shift	Match complaints to trip operations data. Sit with Operations to read cases	P1

Table 9. Ideas to test, in order. The order reflects how much harm is possible and how urgent the moment is. It does not reflect how sure I am of the cause.

Help Center check. The three MoveInSync Help Center pages I read cover login failures, OTP delivery, crashes, launch and loading problems, opt-in and opt-out visibility, and network issues. That shows these problems are known. It does not show how often they happen or how much they hurt the rating.

5. My 30-day execution plan

This is the most important part of the report. I am not promising a big rating jump in 30 days. What I want is to find the failures that matter most, fix one or two of them once they are confirmed, and leave behind a routine the team can keep running each week.

I would follow these rules:

- Prove a problem before building a fix. A review alone is not enough.
- Anything that touches money, safety or a missed shift comes first, even if it happens less often.
- Make small changes that can be undone, and give them to a small group first.
- Watch the early signs, like correct bookings and fresh locations. The star rating moves last.
- Talk to the people who run the service every day. Operations, Support and drivers know things no dashboard shows.

5.1 Who does what

There are five workstreams. Each has one person who is accountable, so nothing falls between teams. In Table 10, A means accountable, R means does the work, C means is asked for input, and I means is kept informed.

Workstream	What it covers	Product	Mobile	Backend	QA	Ops / Support	FDE
WS1 Review analysis	Re-run the notebook, fix tags, check tag accuracy, weekly report	C	I	I	I	C	A / R
WS2 Booking	H1 and H5: audit, tests, fix	A	R	R	R	C	R
WS3 No-show	H2: rules, audit, appeals	A	C	R	R	R	R
WS4 Tracking and alerts	H3 and H4: location age, ETA, alert path	C	R	R	R	C	A
WS5 Operations link	H7: late or changed vehicles, driver issues	C	I	C	I	A	R
WS6 Measuring and release	Numbers, dashboards, safe rollout	C	R	R	C	I	A / R

Table 10. Who is responsible for what. Names get filled in on Day 1.

5.2 The plan, step by step

Each phase has a clear “gate”. We only move on when the gate is met. If it is not, we stop and think again.

Phase and days	What I would do	What we have at the end
Phase 0 Days 1–2 Get aligned and fix the data	Hold a short kick-off with Product, Support, Operations, Mobile and Backend. Agree what “booking mismatch”, “stale location” and “wrong no-show” mean. Share the review themes and ask which ones match cases the team already knows about. Write down the current listing score, total rating count and update date, with the date and time, because my script did not save them. Fix the star-filter call in the notebook (filter_score_with) and re-run the separate one-star and two-star pull. Then check 50 to 100 reviews by hand, across ratings and dates, and measure how often the keyword labels agree with a person.	Shared problem list, agreed definitions, named owners, a verified listing baseline, a larger low-star sample, corrected labels with a measured agreement rate, and a list of missing data. **Gate:** Product and Operations sign off the definitions.
Phase 1 Days 3–5 Trace real cases	Take a small set of real cases, with personal details removed, and follow each one from the employee's action to the saved booking, the driver and vehicle update, and the alert. Start with booking confirmation and no-show disputes, because the cost to the employee can be high. Write the first queries for booking mismatch, location age, ETA error and alert delay, where data exists.	A few cases traced from start to end, first baseline numbers, and a clearer idea of which hypotheses hold. **Gate:** at least one hypothesis is confirmed or rejected with evidence.
Phase 2 Days 6–7 Decide	Go through the evidence with the people closest to the work. Separate app bugs from network, phone, driver and operations causes. Pick the smallest fix with the biggest effect. Agree how we test it, what we watch, and when we roll back. No big redesign.	A short ranked list of work, one or two fixes chosen, tests and rollback rules agreed. **Gate:** Product and Engineering approve the fix.
Phase 3 Week 2 Build	Build the fix behind a feature flag or a staged release. Write tests for the exact failure we saw. Add tracking events so we can tell if the change worked. Set up the dashboard and the comparison of support contacts.	A tested change, a release and rollback plan, and a dashboard showing the result we care about. **Gate:** tests pass and the dashboard shows live data.

Phase and days	What I would do	What we have at the end
Phase 4 Week 3 Release	Release to a small group first. Check every day: booking correctness, no-show disputes, location age, alert delay, crashes and support contacts. Talk to Operations right away if numbers move in a way we did not expect. Write down the decision.	A written decision to expand, adjust or roll back, with the evidence. **Gate:** guardrails held for the agreed watch period.
Phase 5 Week 4 Learn	Compare the numbers after release with the baseline. List other changes that could explain the movement. Share what worked and what did not. Turn the remaining topics into next sprint's work. Hand the weekly review to a permanent owner.	A short readout, a list of next steps and a weekly review that keeps running. **Gate:** leadership readout done.

Table 11. Phases and gates.

5.3 The 30 days at a glance

Workstream	Days 1–2	Days 3–5	Days 6–7	Week 2	Week 3	Week 4
WS1 Review analysis	Fix tags	Check tags	First report	Weekly	Weekly	Hand over
WS2 Booking	Define	Trace cases	Pick fix	Build, test	Small release	Evaluate
WS3 No-show	Define	Trace cases	Pick fix	Check rules	Small release	Evaluate
WS4 Tracking, alerts	Define	Baseline	Rank	Add events	Watch	Evaluate
WS5 Operations link	Meet Ops	Late pickups	Pick action	Agree plan	Watch	Evaluate
WS6 Measuring	Definitions	Queries	Dashboard	Guardrails	Daily watch	Readout

Table 12. Timeline. Booking and no-show go first because the cost to employees is highest.

5.4 What the app and backend need to record

I can't test most of my ideas without a shared record of what happened at each step. I suggest a small set of events. Where the system already logs something, we reuse it, and we only add what is missing. Each event carries a trip or booking ID that hides the person's identity, a server time, a phone time when it matters, the app version and the platform. We do not need names or free text.

Step	Event	Why we need it	Idea
Book	booking_submitted	Records what the employee chose (date, time, type) at the moment they tapped	H1, H5
Book	booking_persisted	Shows what the backend actually saved, to compare with the line above	H1
Book	booking_confirmation_shown	Shows what the employee saw, so we can find confusing screens	H1, H5
Inform	trip_assigned	Links the booking to a vehicle, route and pickup time	H1, H3
Inform	notification_triggered, provider_accepted, displayed	Tells us if the delay is on our server, the push service or the phone	H4
Wait	location_received, location_displayed	Gives the age of the location when the employee looked at it	H3
Wait	eta_computed, arrival_observed	Lets us work out how wrong the ETA was	H3
Board	checkin_recorded	Shows if and when the employee checked in	H2
Finish	noshow_decided (with the rule and inputs)	Lets us explain every no-show decision	H2

Step	Event	Why we need it	Idea
Finish	noshow_disputed, noshow_resolved	Tells us how often no-shows were wrong and how long fixing them takes	H2
Wait	vehicle_assigned, driver_cancelled, vehicle_swapped	Shows how often the vehicle or driver changes after booking	H7
Board	pickup_completed (scheduled and actual time)	Gives pickup punctuality by route and shift	H7
Any time	app_crash, otp_result, login_result, network_error	Baseline for crashes, sign-in and network failures	H6

Table 13. Minimum set of events.

5.5 Releasing safely

A mistake in booking or no-show logic can hurt employees directly, so I would never release a fix to everyone at once. I have left the threshold values empty on purpose. They should come from the baseline numbers in Phase 2. I don't want to guess them now.

Stage	Who gets it	What we watch	Move on if	Roll back if
0. Inside the team	Testers and QA, using made-up trips	Test suite and manual checks	All tests pass	Any confirmed serious mismatch
1. Small group	One site or shift group, agreed with Operations	Daily numbers and support contacts	No guardrail is broken for the agreed period	A guardrail is broken, or Operations raises a safety concern
2. Bigger group	Several sites, or a larger share of users	The same numbers, compared with a group that did not get the change	It keeps improving and nothing gets worse	Worse than the comparison group by more than the agreed limit
3. Everyone	All users	Weekly review, then normal monitoring	Stable for a full week	Any serious mismatch. Rollback is already approved

Table 14. Staged release.

- Numbers that must not get worse: booking mismatch, crash-free sessions, support contacts per thousand trips, no-show disputes and alert delay.
- I would not touch OTP, driver checks or incident escalation without a clear sign-off from Operations and Security.
- One engineering owner decides whether to expand or roll back, and writes the reason in the decision log.

5.6 How I would measure success

The star rating moves last, so I would track the early signs first. For each number, the team should write down who owns it, the query it comes from, the time period, what is left out, and what an alert should trigger. We set targets only after we know the baseline.

Measure	Plain meaning	Comes from	How often
Booking mismatch rate	Bookings where the saved date, time or type is not what the user meant, divided by bookings checked (exclusions written down)	booking_submitted compared with booking_persisted	Daily

Measure	Plain meaning	Comes from	How often
Old-location rate	Tracking events where the location is older than an agreed limit, divided by all relevant tracking events	location_received times	Daily
ETA error	How far the predicted arrival time was from the real one, at an agreed checkpoint	eta_computed and arrival_observed	Daily or weekly
Alert delay	Time from the trigger to the delivery step we can measure. Accepted by the push service does not mean the user saw it	notification events	Daily
Pickup punctuality	Gap between scheduled and actual pickup time, by route and shift	pickup_completed	Daily
Vehicle or driver change rate	Trips where the vehicle or driver changed or was cancelled after assignment, divided by assigned trips	vehicle_assigned, driver_cancelled	Daily
Wrong no-show rate	No-shows found to be wrong after review, divided by no-shows reviewed	noshow_decided plus review	Weekly
Low-star share	New 1-star and 2-star reviews divided by all new reviews in the same period. Read it together with trip numbers	Review notebook	Weekly

Table 15. The measures.

How I would judge the results

- Get a baseline first, ideally several weeks, or as much history as exists.
- Compare like with like: the same days of the week and the same time window, and a comparison group where possible.
- Write down anything else that could move the numbers: other releases, festivals, weather, driver roster changes.
- If a group is small, say so. A result that points in a direction is still useful, but it is not proof.
- Do not chase the star count. It is fine to ask for feedback after a finished trip, within Google's rules. Negative feedback must never be hidden, and nobody should be interrupted while booking, waiting for pickup or dealing with an incident.

5.7 Keeping everyone in step

I would run a 15-minute check-in every week with four questions. What did we learn? What changed for employees? What are we still not sure about? Who owns the next action, and by when? If the evidence does not support a fix, we say so and change the hypothesis. We do not force a fix to fit.

Meeting	When	Who	Purpose
Weekly check-in	Every week, 15 minutes	Product, Mobile, Backend, Ops, FDE	Progress, blockers, next steps
Evidence review	Day 5 and Day 7	Workstream owners, Operations lead	Confirm or drop hypotheses, choose fixes
Release review	Each release	Engineering owner, QA, Operations	Expand, adjust or roll back
Leadership readout	Day 30	Product and Operations leaders	What improved, what did not, what is next

Table 16. Meetings.

Every decision goes into a short log with the date, the evidence, the owner and the result. That way the reasoning is still there when people change roles, and anyone can check it later.

5.8 Risks and how I would handle them

Risk	Chance / impact	What I would do	Owner
Internal data comes late or is incomplete	Medium / High	Start with public data and made-up traces. Ask for limited-access exports on Day 1. Write the gaps down	FDE
Keyword tags label reviews wrongly	High / Medium	Check a sample by hand every week, track the error rate, improve the tags	FDE
A fix affects safety or pay-related flows	Low / Very high	Small groups first, rollback agreed beforehand, sign-off from Operations and Security	Engineering owner
Other releases confuse the before-and-after comparison	Medium / Medium	Mark releases on the timeline, use a comparison group, note other changes	FDE
The scraping tool changes or stops working	Medium / Low	Keep the notebook in version control and test it after every library update. The star filter call already broke once. Use an approved export if one is available	FDE
Privacy concerns with review or trip data	Low / High	Hide identities, keep only what is needed, never send sensitive text to outside AI tools	FDE, Security
Teams feel the plan is blaming them	Medium / Medium	Present findings as ideas to test, share the evidence openly, credit the teams who fix things	FDE, Product

Table 17. Risks.

5.9 What “done” looks like after 30 days

- Shared definitions, and better review tags that a person has checked.
- At least one booking or no-show problem followed from start to end, and either confirmed or ruled out.
- A baseline for each early-sign measure, with the missing data listed honestly.
- One or two fixes released step by step, each with a written decision to expand, adjust or roll back.
- A weekly review routine with a named owner and a decision log.
- A short readout for leaders that separates what we proved, what we suspect and what we still don't know.

6. A weekly routine for “how are our ratings?”

Someone will keep asking this question, so I would turn the answer into a short weekly routine and not a one-time document. It has six steps: collect the new reviews and the listing numbers; clean them (dates, ratings, app version, duplicates); tag them by topic with a person checking some; score each topic by how bad it is, how often it happens and how sure we are, and raise it if it touches safety, money or something that cannot be undone; match topics to trip and support data when we are allowed to; and write a short weekly summary with owners and release notes.

When	What happens	Who
Monday	Record the listing score, rating count and update date with the date and time. Refresh the reviews. Tag new reviews. Compare the last four weeks	Analyst or FDE. Keep the date and app on every record
Tuesday	Look at top topics next to incidents. Give each an owner, priority, due date and next test	Product, Engineering, Support, Operations
Each release	Note the date and version. Compare early-sign numbers and review topics. Watch the guardrails	Engineering owner decides to expand or roll back
Monthly	Sum up what got better, what did not, open risks and ideas for next quarter	Shared with Product and Operations leaders

Table 18. The weekly routine.

Handling the data carefully

- Use public reviews in line with Google Play's terms. If there is an approved API or export, use it instead of scraping.
- Don't send personal or sensitive review text to outside AI services. Remove what is not needed.
- Keep source links, dates, labels and any manual changes, so anyone can check how a review was tagged.
- Treat AI or rule-based tags as suggestions. Track the mistakes and check a sample by hand.
- Make the code handle pages that don't load, duplicates, missing star ratings and empty data.

7. Limits, data request and conclusion

Limits

- The 1,000 reviews are the newest accessible written reviews over 118 days. They are not random, and they are not the whole history.
- The separate one-star and two-star pull failed, so the 292 low-star reviews are the low-star part of the all-rating sample. A re-run is planned for Phase 0.
- The listing score (4.3), the rating count (about 1.15 lakh) and the update date were noted by hand on 10 October 2026. The script did not capture them.
- The theme counts in Table 3 are read from a chart and are approximate. They count mentions, not separate reviews, and the labels have not been checked by a person.
- The per-day rates assume the collection has no gaps within the window.
- Counts have no trip or user numbers behind them, so they are not failure rates.
- Public reviews can be wrong or incomplete. Causes stay unproven until we match them with event logs, support data and operations data.
- I did not invent targets. Thresholds and goals need internal baselines.

What I would ask for on day one

A 90-day review export with dates and ratings, with identities removed; app version and phone details; booking and trip events; GPS and ETA timestamps; push and delay logs; crash-free session numbers; support ticket tags; trip operations data (scheduled and actual pickup times, vehicle and driver changes, cancellations); and no-show disputes with their outcomes. Access should be limited to what is needed, with hidden identifiers and no more personal data than necessary.

Conclusion

The 1,000-review dataset gives a clear direction but not a final answer. Unhappy reviewers write mostly about drivers and vehicles, booking, crashes and network, and tracking. These are moments where the app and the real trip do not match, or where operations let the employee down. Review volume is steady, so there is no sign of a sudden change. My plan is to fix the data gaps first, confirm the failures with real event data, fix the proven ones in small steps, and then measure two things: whether the commute got better, and whether the reviews followed.

References

[1] Google Play, MoveInSync employee app listing (India), package com.moveinsync.ets. Noted by hand on 10 October 2026: 4.3 stars, about 1.15 lakh ratings, updated 8 October 2026.

[2] MoveInSync Help Center, “Mobile App Issues”, last modified 14 May 2025, and two further Help Center pages read during collection.

[3] Candidate assignment and Forward Deployed Engineer job description given during this hiring process.

[4] Review collection notebook (Google Colab), MoveInSync_Review_Collection_and_Analysis.ipynb; newest accessible reviews, India store, English, up to 1,000 records: https://colab.research.google.com/drive/1fP52lR-nGd6N0qryWATvWfgnzOhGsO0j?usp=sharing